pandas basics

Now that you have a good understanding of the core structures and routines of Python, and some of the basics of NumPy, you're ready to start working with pandas. Pandas is one of the primary tools that you'll use throughout the rest of this certificate program, as well as in a large and growing number of data professions. In this video, you'll learn about the main classes in the pandas library and some important ways to work with them. Pandas has two core object classes: dataframes and series. Let's begin with a review of dataframes. A dataframe is a two-dimensional, labeled data structure with rows and columns. You can think of a dataframe like a spreadsheet or a SQL table.
It can contain many different kinds of data. Data professionals use dataframes to structure, manipulate, and analyze data in pandas, just like we did in the previous video with the Titanic example. We can create a dataframe using the pandas DataFrame function. This function has a lot of flexibility, and can convert numerous data formats to a DataFrame object. In this example, we created a dataframe from a dictionary, where each key of the dictionary represents a column name, and the values for that key are in a list. Each element in the list represents a value for a different row at that column. We can also create one from a NumPy array resembling a list of lists, where each sub-list represents a row of the table.

Notice that in this example, we included separate keyword arguments for columns and index. This approach lets us name the columns and rows of the dataframe. These are just a couple of the many different ways to create a dataframe with the DataFrame function. For examples of some others, be sure to review the available pandas documentation on this topic. Often, data professionals need to be able to create a dataframe from existing data that's not written in Python syntax. For example, maybe we want to take an existing spreadsheet and manipulate it in pandas. Spreadsheets can be saved as CSV files, which can then be read into pandas as a dataframe.

CSV stands for comma-separated values, and it refers to a plaintext file that uses commas to separate distinct values from one another. Here is a sample of the first few lines of source data from the Titanic dataset that we used previously. This is what a CSV file looks like. In this file, you'll find values for passenger name, age, sex, fare, and more. Notice that a comma is used to separate each value from the next. To create a dataframe from a CSV file. pandas has the "read CSV" function. Here's the same Titanic data rendered as a dataframe.

For the sake of an example, it's defined here as df3. The "read CSV" function can read files from a URL, like in this example, and it can also read files directly from your hard drive. Instead of a URL, you'd just provide the file path to your file. Now, let's discuss the other main class in pandas: Series. A Series is a one-dimensional, labeled array. Series objects are most often used to represent individual columns or rows of a dataframe. So, if we select a row or a column from this Titanic dataframe and call "type" on it, it will return as a pandas series object.

Like dataframes, individual series can be created from various data objects, including from NumPy arrays, dictionaries, and even scalars. Again, refer to the pandas documentation for examples. Now, let's use the Titanic dataset to review some of the basics of working with dataframes and series. The DataFrame and Series classes have many super useful methods and attributes that make common tasks easier. Remember, a method is a function that belongs to a class. It performs an action on the object. An attribute is a value associated with a class instance.

It typically represents a characteristic of the instance. Both methods and attributes are accessed using dot notation, but methods use parentheses, while attributes do not. Earlier in the video, we named the Titanic dataset "df3," but let's change the name to "titanic" for clarity. We can do this by simple reassignment. If we want to access the "columns" of the dataframe, we can use the columns attribute. This returns an index of all of the column names. We can use the shape attribute to check the number of rows and columns contained in the dataframe.

This dataframe has 891 rows and 12 columns. And we can get some summary information about the dataframe by calling the info method. This tells us that there are 891 rows and 12 columns, and it also gives us the column names, the data type contained in each column, the number of non-null values in each column, and the amount of memory the dataframe uses. By the way, I want to address a couple of points about terminology in pandas. First, null values in pandas are represented by NaN, which stands for "not a number." And second, if a Series object contains mixed or string data types, when you check its data type, it will come back as an "object." This is an

example of how pandas is built on NumPy, but the details of this are beyond the scope of this video.

One of the most common tasks when working in pandas is selecting or referencing parts of the dataframe. This has many similarities with indexing and slicing. For example, if you want to select a single column, you can type the name of the dataframe followed by brackets, and within the brackets enter the name of the column as a string. This returns a Series object of that column. You can also use dot notation, but this only works if the column name does not contain any whitespaces. Using dot notation is faster to type, so for very simple lines of code, you may prefer to do this. But if the code begins to get more complex, it's generally better to use bracket notation, because it makes the code easier to read.

To select multiple columns of a dataframe by name, use bracket notation. Within the brackets, enter a list of column names. This returns a view of your dataframe as a new DataFrame object. If you want to select rows or columns by index, you'll need to use iloc. iloc is a way to indicate in pandas that you want to select by integer-location-based position. If you enter a single integer into the iloc brackets, you'll get a series object representing a single row of your dataframe at that index. Because I entered 0 here, I got the very first row in my dataframe as a series. If you enter a LIST of a single integer in the iloc brackets, you'll get a DataFrame object of a single row of the dataframe at that index.

You can access a range of rows by entering the indices of the beginning and ending rows separated by a colon. Pandas will return every index starting with the beginning index up to, but not including, the last index. So zero colon three returns row indices 0,

1, and 2. You can select subsets of rows and columns together, too. This returns a dataframe view of rows 0, 1, and 2 at columns 3 and 4 only. So, if you want a single column in its entirety, you select all rows, and then enter the index of the column you want. And, you can even get a single value at a particular row in a particular column by using two indices separated by a comma.

Loc is similar to iloc, but instead of selecting by index location, loc is used to select pandas rows and columns by name. Let's investigate loc with the Titanic dataframe... In this example, I'm selecting rows 1, 2, and 3 at just the "Name" column. Note that in this example, we're referring to the rows with numbers, even though we're using loc to select. This is because our rows are indexed by number. If we had a named index, however, we'd have to use row names, like what we're doing for columns. And one more thing.

If you want to add a new column to a dataframe, you can do that with a simple assignment statement. Now we have a new column at the end here. There are so many things you can do in pandas, and I can only share so much in the time we have. As always, the documentation is your friend. There will inevitably be times where you need to do something that wasn't explicitly covered here. In those cases, the documentation almost always has simple examples that demonstrate how to do the thing you need to do. There's still more to come though, so I'll meet you soon.

Core pandas object classes

- DataFrame
- Series

DataFrame

A two-dimensional, labeled data structure with rows and columns

CSV file

Stands for “comma-separated values.” A plaintext file that uses commas to separate distinct values from one another

Question

Fill in the blank: In pandas a dataframe is a _____-dimensional, labeled data structure.

- one
- three
- two
- zero

Correct
In pandas a dataframe is a two-dimensional, labeled data structure. A dataframe is organized into rows and columns.

Continue Skip

Series

A one-dimensional, labeled array

NaN

How null values are represented in pandas, which stands for “not a number”

`iloc[]`

A way to indicate in pandas that you want to select by integer-location-based position

`loc[]`

Used to select pandas rows and columns by name